FIELD PROGRAMME RAHIM – WEEK 8

Real-Time System Hardening, Data Quality & Deployment Readiness

By Friday, you should be able to:

- Explain the complete data flow from simulated motor sensors to the backend, ML model and dashboard.
- Validate incoming sensor data and handle missing, invalid, duplicated or delayed readings.
- Use timestamps and sequence information to reason about data freshness and ordering.
- Improve the backend by separating data validation, storage and prediction responsibilities.
- Understand API error handling, logging and basic system health monitoring.
- Test the system using healthy, warning, fault and abnormal communication scenarios.
- Document how the prototype could transition from a Python simulator to a physical ESP32.

Week 8 focus: Week 7 established the end-to-end prototype. This week is about making that prototype dependable and testable—not adding unnecessary complexity.

Day 1 – Review the End-to-End System & Data Quality

Goal
Understand what happens to a sensor reading as it travels through the system, and identify where bad data can enter.

Learn

- Review the Week 7 architecture: simulator → HTTP/JSON → FastAPI → storage → ML prediction → Streamlit dashboard.
- Understand data validation, required fields, data types, valid ranges and timestamps.
- Understand the difference between a missing reading, an invalid reading and a stale reading.
- Review HTTP status codes: 200, 201, 400, 422 and 500.

Practical Work

Program 1 – Inspect the Current System

- Run the simulator, backend and dashboard together.
- Trace one sensor reading from generation to dashboard display.
- Record the data format used by the simulator and the fields expected by the backend.
- Identify at least three possible failure points.

Program 2 – Validate Incoming Sensor Data
Add validation to the API for required sensor fields such as temperature, vibration, current, sound and RPM.

- Reject missing required fields.
- Reject values with the wrong type or impossible format.
- Handle negative or out-of-range values according to documented prototype assumptions.
- Return a clear response when the payload is invalid.

Deliverable

- Create **data_validation_notes.md** describing the payload, validation rules, and examples of accepted and rejected readings.

- Update the backend validation and demonstrate at least three invalid payloads.

Git Task

Commit and push: **Week 8 Day 1 – Sensor Data Validation**

Day 2 – Reliable API Communication & Error Handling

Goal
Make the simulator communicate with the backend reliably and understand what happens when communication fails.

Learn

- HTTP request/response cycle and API response codes.
- Connection timeout, server unavailable and failed requests.
- Retry strategy and why unlimited retries are dangerous.
- Idempotency and the risk of storing the same reading more than once.
- Basic logging: timestamp, event, level and useful context.

Practical Work

Program 1 – Improve the Simulator Client

- Send sensor data to the API using HTTP POST.
- Set a reasonable request timeout.
- Handle connection errors and non-success HTTP responses.
- Add a limited retry strategy with a short delay.
- Print a clear message for success, retry and final failure.

Program 2 – Add a Reading Identifier
Add a unique reading ID or sequence number to each generated payload. Use it to identify repeated submissions.

```
{
  "reading_id": "motor-01-000125",
  "timestamp": "2026-11-02T10:00:03Z",
  "temperature": 61.5,
  "vibration": 0.17,
  "current": 8.3,
  "sound": 32,
  "rpm": 1481
}
```

Program 3 – Test Communication Failures

- Stop the backend while the simulator is running.
- Observe the client response and restart the backend.
- Confirm the simulator recovers without crashing.
- Check whether repeated readings are stored more than once.

Deliverable

- Update **motor_sensor_simulator.py** with timeout and error handling.
- Create **communication_test_notes.md** with test cases, expected behavior and observed results.

Git Task

Commit and push: **Week 8 Day 2 – API Reliability**

Day 3 – Sensor History, Freshness & Storage

Goal
Ensure the system stores sensor readings consistently and can distinguish current data from old data.

Learn

- Why timestamps matter in real-time monitoring.
- UTC timestamps versus local display time.
- Out-of-order and delayed sensor readings.
- Data freshness: when the latest reading was received versus when it was generated.
- CSV/SQLite storage trade-offs for a small prototype.

Practical Work

Program 1 – Store Consistent Records

- Store the reading ID, sensor values, source timestamp and server-received timestamp.
- Keep the data format consistent across records.
- Use CSV or SQLite; do not introduce a complex database without a clear need.

Program 2 – Detect Stale Data

- Track the timestamp of the most recent reading.
- Define a prototype freshness limit, for example 10 seconds, and document that it is a test value.
- Display whether data is LIVE or STALE.
- Test by stopping the simulator and observing the status.

Program 3 – Review the Stored Data

- Verify that readings have timestamps and unique IDs.
- Check for duplicate IDs and missing fields.
- Confirm that the stored history can be loaded for plotting.

Deliverable

- Create or update **sensor_data.csv** or the SQLite database.
- Create **data_storage_notes.md** explaining the schema and stale-data behavior.

Git Task

Commit and push: **Week 8 Day 3 – Sensor History & Freshness**

Day 4 – System Testing & Dashboard Improvements

Goal
Test the integrated system under normal and abnormal conditions and make the dashboard communicate system status clearly.

Learn

- Unit testing versus integration testing.
- Test cases, expected results and observed results.
- Sensor status versus system status.
- Why an ML prediction is not a certified safety decision.

- How to show stale or unavailable data without misleading the user.

Practical Work

Program 1 – Create an Integration Test Checklist

- Valid sensor payload is accepted and stored.
- Invalid payload is rejected with a useful response.
- Simulator reconnects after backend downtime.
- Duplicate reading ID is handled consistently.
- Dashboard updates when new readings arrive.
- Dashboard indicates stale data when readings stop.

Program 2 – Improve the Dashboard

- Show latest temperature, vibration, current, sound and RPM.
- Show the latest reading timestamp and data freshness status.
- Show motor health prediction and confidence if the model provides it.
- Show a clear message when no recent data is available.
- Keep recent history and graphs readable.

Program 3 – Scenario Testing

- Run HEALTHY mode and record the dashboard behavior.
- Run WARNING mode and observe gradual changes.
- Run FAULT mode and observe the prediction and alert.
- Stop the simulator and confirm the dashboard does not continue presenting old readings as live.

Deliverable

- Create **week8_test_cases.md** with test, expected result, actual result and pass/fail.
- Update **dashboard.py** to show freshness and system status.

Git Task

Commit and push: **Week 8 Day 4 – Integration Testing & Dashboard**

Day 5 – System Integration, Documentation & Demonstration

Goal
Demonstrate a dependable prototype and explain what remains before using real ESP32 hardware.

Practical Work

Program 1 – Run the Complete System

```
Motor Sensor Simulator
        ↓ HTTP / JSON
      FastAPI Backend
        ↓ validate + timestamp
      Storage / Sensor History
        ↓
      Week 6 ML Model
        ↓
      Streamlit Dashboard
```

Program 2 – Prepare a Run Guide

- Document the required Python version and dependencies.

- Document how to start the backend, simulator and dashboard.
- Document the API endpoint and example payload.
- Document how to switch between HEALTHY, WARNING and FAULT modes.
- Document known limitations and troubleshooting steps.

Program 3 – ESP32 Transition Notes

- Identify which part of the Python simulator would be replaced by the ESP32.
- Explain how real sensors connect to the ESP32 and how readings are collected.
- Explain how the ESP32 could send the same data fields over Wi-Fi.
- Identify what should remain unchanged: backend validation, storage, ML inference and dashboard, subject to interface compatibility.
- List hardware details that still need confirmation, such as sensor model, sampling rate and calibration.

Friday Demonstration

1. Start the backend and dashboard.
2. Start the simulator in HEALTHY mode and show incoming readings.
3. Switch to WARNING and show readings changing over time.
4. Switch to FAULT and show the model result and alert.
5. Stop the simulator and show the dashboard marking data as STALE or unavailable.
6. Send an invalid payload and explain how the backend handles it.
7. Explain how the simulator could be replaced with a physical ESP32.

Friday Questions

- Why should the backend validate data even if the simulator already validates it?
- What is the difference between a reading timestamp and a received timestamp?
- How can the system identify stale data?
- What happens if the backend is temporarily unavailable?
- How do we avoid processing a repeated reading twice?
- Why must a dashboard distinguish a motor fault from a communication failure?
- Which components would change when real ESP32 hardware is introduced?
- Why is this prototype not a certified industrial safety system?

Week 8 Deliverables

- Updated simulator with timeout, retry and reading identifier.
- Backend with payload validation, timestamps and clear error responses.
- Consistent sensor history stored in CSV or SQLite.
- Dashboard displaying readings, health prediction and data freshness.
- Integration test checklist with observed results.
- README.md with setup, run instructions, architecture and limitations.
- All work committed and pushed to the repository.

Suggested Repository Structure

```
python-learning/
```

```
week7/
    simulator/
    backend/
    ml/
    dashboard/
    data/
week8/
    simulator/
    motor_sensor_simulator.py
    backend/
    main.py
    dashboard/
    dashboard.py
    data/
    sensor_data.csv
    tests/
    week8_test_cases.md
    docs/
    data_validation_notes.md
    communication_test_notes.md
    data_storage_notes.md
    README.md
    requirements.txt
```

Important: Week 8 is a prototype hardening and testing week. The freshness limit, retry behavior and validation ranges are learning assumptions unless confirmed by actual motor and sensor specifications. The ML output should be treated as an experimental prediction, not a safety guarantee.